

Wanna Try V1 — Complete Technical Architecture PRD

Document Type: Technical Architecture & Engineering PRD

Product: Wanna Try

Version: V1.0

Status: Architecture Baseline

Date: September 2026

1. Executive Summary

Wanna Try is a personalized travel and experience-discovery platform combining:

- Personalized onboarding
- AI-powered recommendations
- Personalized content feed
- Travel and experience discovery
- Trip planning
- Itinerary management
- Maps and location services
- AI assistant
- Voice interaction
- Reminders
- Booking and affiliate integrations
- Contextual advertising
- User behavioral intelligence
- Personalized recommendations

The first production version will use a **modular-monolith architecture** for the core backend rather than immediately adopting a large microservice architecture.

The platform will consist of:

1. **Flutter Mobile Application**
2. **Java + Spring Boot Core API**
3. **Python + FastAPI AI/Voice platform**
4. **PostgreSQL + PostGIS**
5. **Redis**
6. **Cloudinary for media**
7. **AWS infrastructure**
8. **Optional MongoDB only when a justified use case appears**

The architecture must support rapid development now while allowing selected modules to become independent services later.

Core principle

Build simple enough to move fast, but structure every major domain so it can be extracted later without a major rewrite.

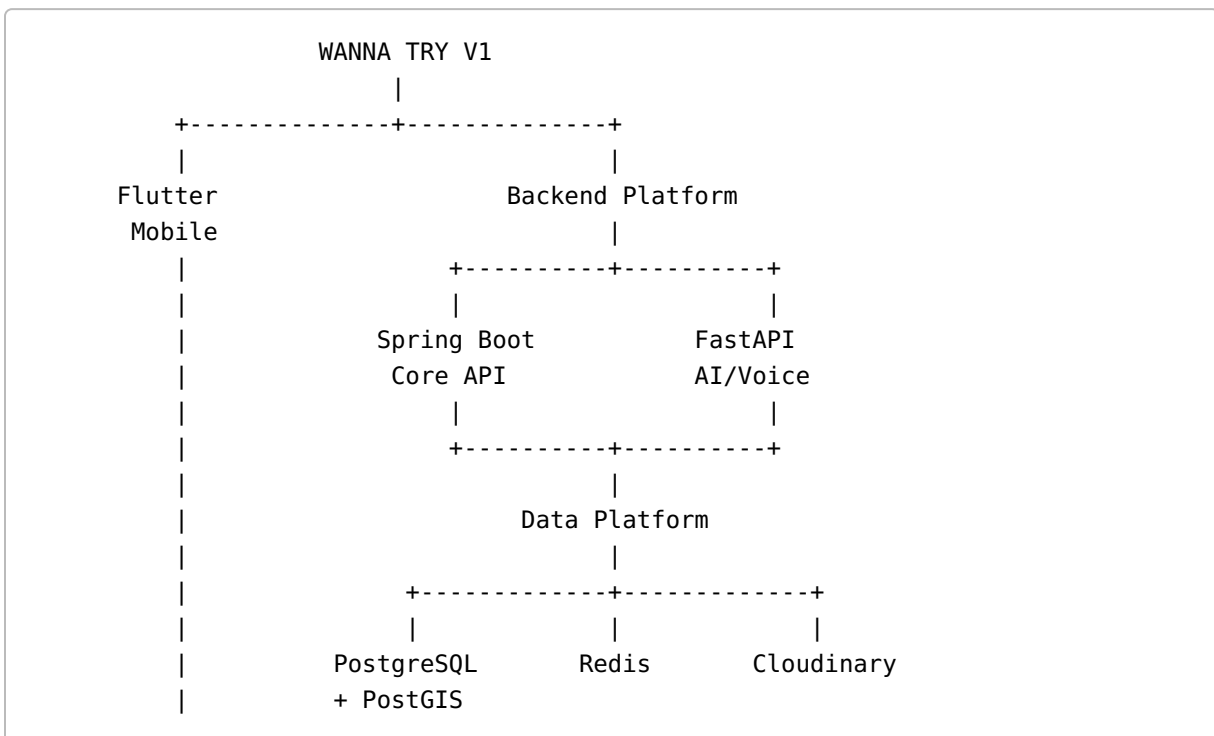
2. Architecture Philosophy

Wanna Try will follow:

Modular Monolith → Selective Service Extraction → Microservices only where justified

The initial system must NOT be designed as 10–20 independently deployed services.

Instead:



3. System Components

The platform has four primary engineering layers.

Layer 1 — Client

Flutter mobile application.

Responsibilities:

- UI
 - Navigation
 - User interactions
 - Local state
 - Local caching
 - Push notification handling
 - Location permissions
 - Maps presentation
 - Voice interface
 - API communication
-

Layer 2 — Core Backend

Java + Spring Boot.

Responsibilities:

- Authentication
 - Users
 - Profiles
 - Onboarding
 - Preferences
 - Experiences
 - Feed
 - Trips
 - Itineraries
 - Reminders
 - Bookings
 - Payments
 - Notifications
 - Ads
 - Business rules
 - API orchestration
-

Layer 3 — AI Platform

Python + FastAPI.

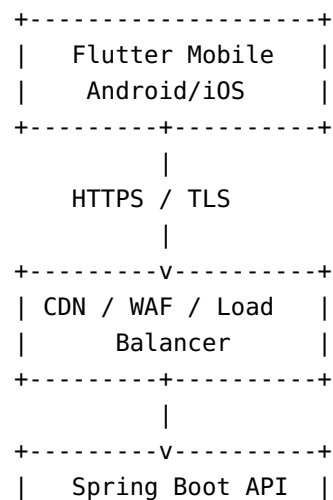
Responsibilities:

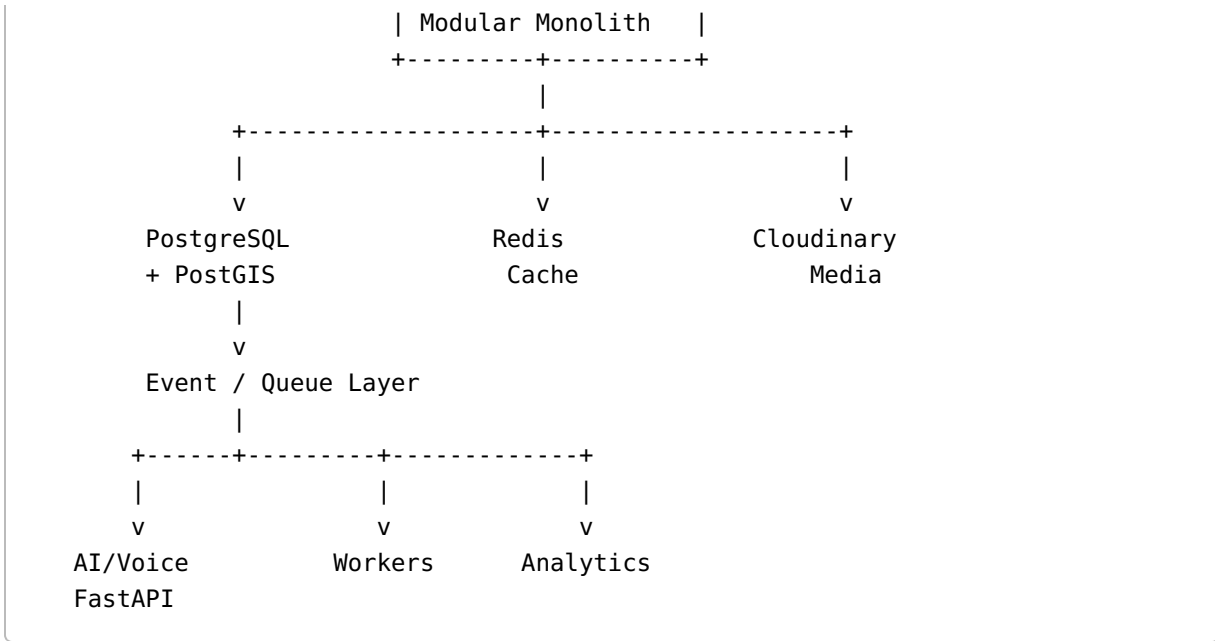
- AI assistant
- Recommendation intelligence
- Embeddings
- Semantic understanding
- AI agents
- RAG
- Ranking models
- Voice orchestration
- Speech-to-text
- Text-to-speech
- AI memory/context

Layer 4 — Data & Infrastructure

- PostgreSQL
 - PostGIS
 - Redis
 - Cloudinary
 - AWS
 - Background workers
 - Event/queue infrastructure
 - Monitoring
 - Logging
 - Analytics
-

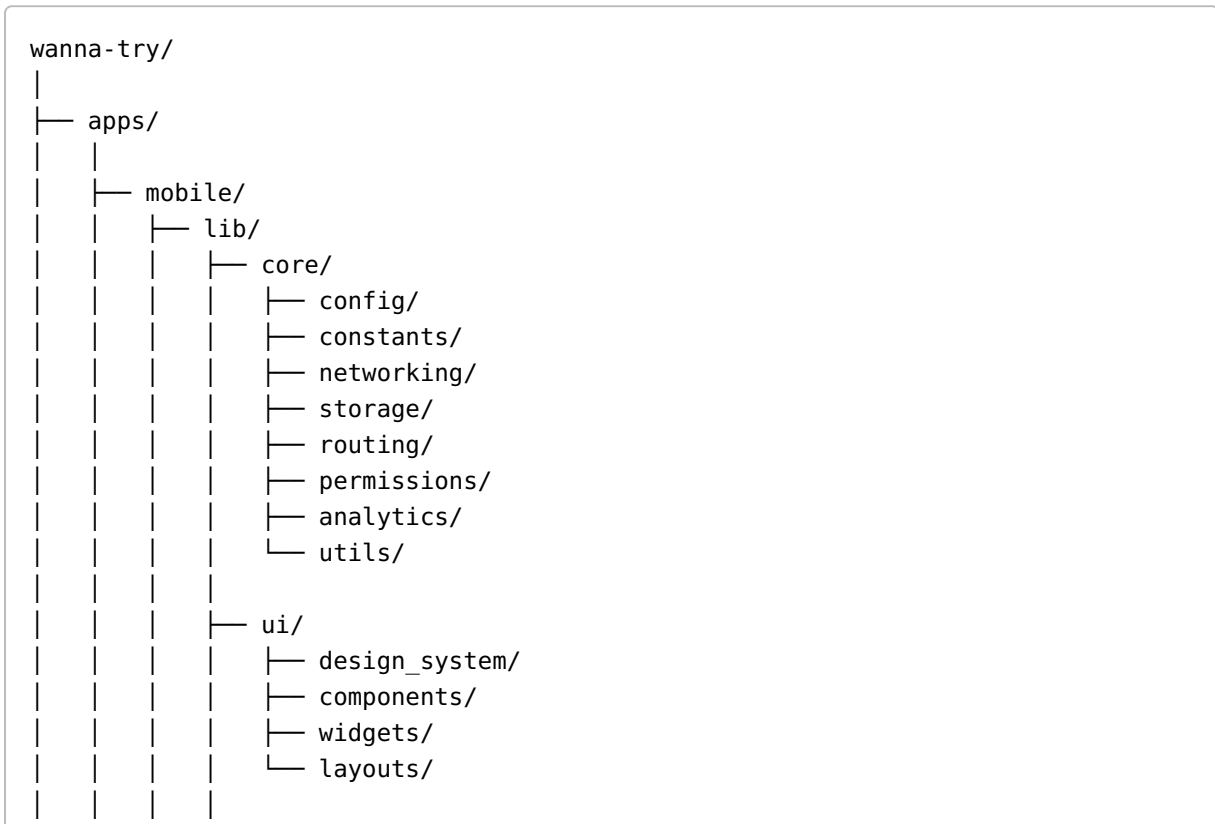
4. High-Level Architecture





5. Repository Architecture

The project should use a monorepo initially.



```
|
|
|
|   └─ features/
|       └─ auth/
|       └─ onboarding/
|       └─ home/
|       └─ feed/
|       └─ experiences/
|       └─ search/
|       └─ places/
|       └─ trips/
|       └─ itinerary/
|       └─ maps/
|       └─ reminders/
|       └─ ai/
|       └─ voice/
|       └─ bookings/
|       └─ profile/
|       └─ settings/
|
|   └─ main.dart
|
|   └─ android/
|   └─ ios/
|   └─ test/
|
|   └─ admin/
|
|   └─ services/
|       └─ core-api/
|           └─ Spring Boot
|
|       └─ ai-service/
|           └─ FastAPI
|
|       └─ voice-service/
|           └─ FastAPI
|
|       └─ workers/
|
|   └─ database/
|       └─ migrations/
|       └─ seeds/
|       └─ documentation/
|
|   └─ infrastructure/
|       └─ docker/
|       └─ aws/
|       └─ terraform/
```

```
|   └─ monitoring/
|
|─ docs/
|   └─ architecture/
|       └─ api/
|           └─ database/
|               └─ ai/
|                   └─ deployment/
|
|─ README.md
```

6. Mobile UI Architecture

Flutter will be the primary mobile technology.

Target platforms:

- Android
- iOS

The UI must have three major levels.

Design System

```
ui/design_system/

colors
typography
spacing
radius
shadows
animations
icons
theme
```

Reusable Components

```
ui/components/

ExperienceCard
DestinationCard
TripCard
```

```
RecommendationCard
SearchBar
FilterChip
Rating
PriceBadge
SaveButton
VoiceButton
ReminderCard
MapCard
```

Feature UI

Each feature owns its screens and feature-specific widgets.

Example:

```
features/feed/

presentation/
  pages/
  widgets/
  controllers/

domain/
  models/
  repositories/

data/
  api/
  dto/
  repository_impl/
```

Dependency direction:

```
UI
↓
Controller / State
↓
Repository
↓
API
↓
Spring Boot
```


7. Mobile State Management

The application should use a consistent state-management solution.

Recommended options:

- Riverpod
- Bloc

The architecture should avoid business logic directly inside widgets.

Widgets should primarily:

- render state
 - trigger actions
 - respond to navigation/events
-

8. Mobile Local Storage

Use local storage for:

- authentication/session metadata
- cached preferences
- onboarding state
- recent searches
- lightweight feed cache
- offline trip drafts
- application settings

Sensitive tokens must use secure platform storage.

9. Backend Technology

Primary Backend

```
Java
Spring Boot
Spring Security
Spring Data JPA
Hibernate
```

Bean Validation
OpenAPI

The core backend is a **modular monolith**.

10. Core Backend Modules

```
core-api/  
  
auth/  
users/  
onboarding/  
preferences/  
feed/  
recommendations/  
experiences/  
places/  
search/  
trips/  
itinerary/  
reminders/  
notifications/  
bookings/  
payments/  
ads/  
analytics/  
integrations/
```

Each module should contain its own:

```
controller  
service  
domain  
repository  
dto  
mapper  
events
```

11. Module Boundary Rules

Modules must not freely access another module's internal implementation.

Bad:

```
FeedService
  ↓
TripRepository
```

Preferred:

```
FeedService
  ↓
TripModule interface
  ↓
TripService
```

This ensures modules can later become independent services.

12. Database Architecture

Primary Database

PostgreSQL.

PostgreSQL is the system of record for transactional product data.

Primary domains:

```
users
profiles
preferences
onboarding
experiences
places
trips
itineraries
reminders
bookings
```

```
payments
ads
```

13. PostGIS

PostGIS will support geographic operations.

Use cases:

- Nearby experiences
- Nearby destinations
- Distance calculation
- Radius searches
- Geo filtering
- Location-based recommendations
- Map-related backend operations

Example:

```
Find experiences
within X kilometers
of user location
```

14. MongoDB Policy

MongoDB is **optional**.

It must NOT be introduced merely because Wanna Try is expected to scale.

PostgreSQL remains the default source of truth.

MongoDB may be introduced later if there is a demonstrated requirement such as:

- document-heavy AI context
- large flexible AI artifacts
- high-volume semi-structured documents
- a specific workload where MongoDB provides a material advantage

Architecture decision:

No MongoDB dependency in the initial MVP unless a concrete requirement is approved.

15. Redis

Redis is the primary caching and low-latency data layer.

Use cases:

```
Feed cache
Recommendation cache
Session-related data
Rate limiting
OTP throttling
Popular content
Search cache
Temporary AI state
Distributed locks
Short-lived tokens
```

Redis must not become the authoritative database for important business records.

16. Cloudinary

Cloudinary will handle application media.

Use cases:

- Experience images
- Destination images
- User-uploaded images
- Profile images
- Optimized thumbnails
- Image transformations
- CDN delivery

Architecture:

```
Flutter
↓
Upload authorization
↓
Cloudinary
↓
```

```
CDN delivery
↓
Flutter
```

Application servers should not unnecessarily proxy large media files.

17. AI Platform

Python + FastAPI will handle AI functionality.

```
Spring Boot
|
v
Python FastAPI
|
+-- LLM
+-- Embeddings
+-- RAG
+-- Recommendation
+-- Ranking
+-- AI Agents
+-- AI Memory
```

The AI layer should not directly own core transactional business logic.

It should request actions from the core backend through controlled APIs/tools.

18. AI Agent Architecture

The AI assistant should operate through controlled tools.

Example tools:

```
searchExperiences()
searchPlaces()
getPlaceDetails()
getDirections()
createTrip()
updateTrip()
addTripActivity()
```

```
createReminder()  
searchBookings()
```

Flow:

```
User  
↓  
AI  
↓  
Intent understanding  
↓  
Tool selection  
↓  
Spring Boot API  
↓  
Business validation  
↓  
Result  
↓  
AI response
```

AI must not bypass business rules.

19. Voice Architecture

Voice should be implemented as a dedicated FastAPI component.

```
Flutter  
|  
| streaming  
v  
Voice Gateway  
|  
+-- Speech-to-Text  
|  
+-- AI Orchestrator  
|  
+-- Tool Calling  
|  
+-- Text-to-Speech
```

Voice should support:

- voice queries
- conversational interaction
- trip planning
- destination discovery
- experience discovery
- reminders
- AI actions

Voice infrastructure should be independently scalable from normal API traffic.

20. Feed Architecture

The feed is one of the most important product systems.

Initial:

```
GET /feed
  ↓
Feed Module
  ↓
Candidate retrieval
  ↓
Personalization
  ↓
Ranking
  ↓
Redis
  ↓
Mobile
```

The feed must avoid expensive multi-table computation on every request.

21. Recommendation System

The recommendation system should evolve over time.

Initial signals:

Onboarding preferences
Destination preferences
Budget
Travel style
Trip duration
Activity preferences

Behavioral signals:

Views
Clicks
Saves
Shares
Dismissals
Searches
Trip creation
Bookings
Reminders
Voice queries

Pipeline:

```
graph TD
    A[Onboarding] --> B[Initial User Profile]
    B --> C[Initial Recommendations]
    C --> D[Feed]
    D --> E[User Behavior]
    E --> F[Events]
    F --> G[Updated Profile]
    G --> H[Improved Recommendations]
```

This creates the Wanna Try personalization loop.

22. User Event System

Important actions should generate events.

Examples:

```
USER_REGISTERED
ONBOARDING_COMPLETED
EXPERIENCE_VIEWED
EXPERIENCE_SAVED
EXPERIENCE_SHARED
EXPERIENCE_DISMISSED
SEARCH_PERFORMED
PLACE_VIEWED
TRIP_CREATED
TRIP_UPDATED
BOOKING_STARTED
BOOKING_COMPLETED
REMINDER_CREATED
VOICE_QUERY
AI_QUERY
AD_IMPRESSION
AD_CLICK
```

Events support:

- analytics
- recommendations
- AI personalization
- notifications
- business intelligence

23. Event Architecture

Initial:

```
Spring Boot
  ↓
Event Publisher
  ↓
Managed Queue
```

↓
Workers

Possible technologies:

- RabbitMQ
- AWS SQS
- equivalent managed queue

Later, if scale requires high-throughput streaming:

Kafka

Kafka is not mandatory for MVP.

24. Background Workers

Background workers process non-critical synchronous workloads.

Examples:

```
Generate recommendations
Generate embeddings
Search indexing
Send notifications
Process media
AI summarization
Analytics processing
Affiliate event processing
```

Pattern:

```
API Request
  ↓
Queue
  ↓
Worker
  ↓
Processing
```

25. Search Architecture

Initial search can use PostgreSQL.

When search volume and complexity justify it:

```
PostgreSQL
  ↓
Indexing pipeline
  ↓
OpenSearch
  ↓
Search API
```

Search can eventually combine:

```
Keyword
+
Location
+
Filters
+
Semantic similarity
+
Personalization
```

26. Maps Architecture

Use a maps provider abstraction.

Core interface:

```
MapsProvider

getPlace()
searchPlaces()
geocode()
reverseGeocode()
```

```
getDirections()  
calculateDistance()
```

This prevents vendor-specific logic from spreading throughout the application.

Initial provider:

Google Maps Platform

Another provider can be introduced later without rewriting product modules.

27. Trip Architecture

Trip domain:

```
Trip  
├─ Destination  
├─ Start Date  
├─ End Date  
├─ Travelers  
├─ Budget  
├─ Trip Days  
│   ├── Activities  
│   ├── Places  
│   ├── Experiences  
│   └─ Notes  
└─ Reminders
```

Trip APIs should support:

```
Create trip  
Update trip  
Delete trip  
Add destination  
Add experience  
Add activity  
Reorder itinerary  
Create reminder  
Share trip
```

28. Reminder Architecture

Reminder types:

```
One-time
Recurring
Trip-based
Activity-based
Booking-based
Location-based
```

Example:

```
Trip
↓
Day 2 activity
↓
Reminder
↓
Scheduler
↓
Notification
```

29. Notification Architecture

Notifications must be asynchronous.

```
Business Event
↓
Queue
↓
Notification Worker
↓
+-----+
|       |
| FCM   | APNs  | Email/SMS
```

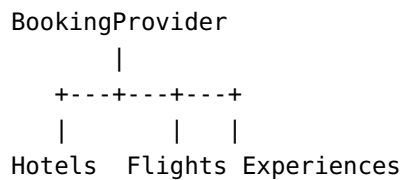
Examples:

- Trip reminder

- Booking update
- New recommendation
- Price update
- Activity reminder

30. Booking Architecture

Create a provider abstraction.



External providers should not be embedded directly throughout the core code.

31. Payments

Payment processing should be isolated behind a payment abstraction.

Potential providers:

- Razorpay
- Stripe
- other region-specific providers

Payment data should be treated as highly sensitive.

Wanna Try should avoid storing raw card details.

32. Ads Architecture

Ads should be context-aware.

Example:

```
User searches
"Kedarkantha trek"
```



Track:

impression
click
conversion
revenue

Ads must not compromise the primary recommendation experience.

33. Authentication

Recommended:

OAuth 2 / OIDC
JWT access tokens
Refresh tokens
Spring Security

Potential login providers:

- Google
- Apple
- Email/password
- phone/OTP where required

Passwords must use secure hashing.

34. Security

Minimum requirements:

```
TLS
WAF
Rate limiting
Input validation
Authentication
Authorization
RBAC
Secrets management
Encryption at rest
Encryption in transit
Audit logs
Secure token storage
```

Secrets must never be committed to Git.

Use AWS Secrets Manager or equivalent.

35. API Architecture

REST APIs initially.

Base:

```
/api/v1/
```

Examples:

```
/api/v1/auth
/api/v1/users
/api/v1/onboarding
/api/v1/preferences
/api/v1/feed
/api/v1/experiences
/api/v1/places
/api/v1/search
/api/v1/trips
/api/v1/reminders
```

```
/api/v1/bookings
/api/v1/payments
/api/v1/notifications
/api/v1/ai
/api/v1/voice
```

Requirements:

- OpenAPI documentation
- DTOs
- Validation
- Consistent errors
- Pagination
- Cursor-based feed pagination
- Idempotency for important operations
- Versioning

36. Feed Pagination

Feed endpoints should use cursor pagination.

Preferred:

```
GET /feed?cursor=abc123
```

Response:

```
{
  "items": [],
  "nextCursor": "xyz456"
}
```

Avoid offset pagination for large personalized feeds.

37. Media Upload

Large files should bypass the application server wherever possible.

```
Mobile
↓
Signed upload
↓
Cloudinary
↓
Media URL
↓
Spring Boot
```

The backend stores media metadata and references, not unnecessary binary payloads.

38. Infrastructure

Initial cloud:

AWS

Recommended initial services:

```
CloudFront
WAF
Load Balancer
ECS
RDS PostgreSQL
ElastiCache Redis
S3
Secrets Manager
CloudWatch
ECR
```

Cloudinary remains the media platform.

39. Containers

Dockerize:

```
core-api
ai-service
```

voice-service
workers

Local development:

Docker Compose

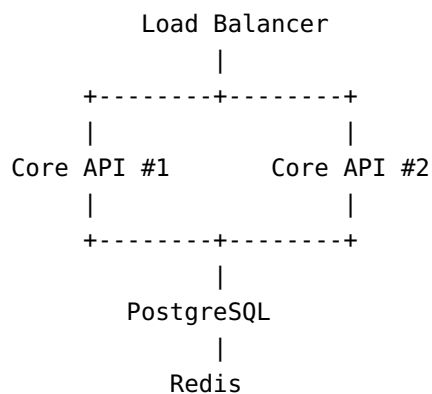
Production:

AWS ECS

Kubernetes/EKS should only be introduced when justified by scale or operational requirements.

40. Deployment Architecture

Initial:



Application instances should be stateless.

41. Environments

Maintain:

local
development

staging
production

Local:

Docker Compose

Production:

AWS

42. CI/CD

Recommended:

```
Developer
↓
Git Push
↓
Pull Request
↓
Unit Tests
↓
Integration Tests
↓
Security Scan
↓
Docker Build
↓
Container Registry
↓
Staging
↓
Validation
↓
Production
```

Technology:

- GitHub
- GitHub Actions

- Docker
 - AWS ECR
 - AWS ECS
-

43. Testing

Backend

```
JUnit
Mockito
Testcontainers
REST Assured
```

Mobile

```
Flutter Unit Tests
Widget Tests
Integration Tests
```

Performance

```
k6
```

Critical integration tests should run against real containerized PostgreSQL/Redis rather than mocks wherever appropriate.

44. Observability

Use:

OpenTelemetry

Track:

```
Request rate
Latency
Error rate
```

- CPU
- Memory
- Database connections
- Redis hit ratio
- Queue depth
- AI latency
- Voice latency
- External API latency

45. Logging

Use structured logs.

Example:

```
{
  "requestId": "...",
  "userId": "...",
  "service": "feed",
  "operation": "getFeed",
  "latency": 120,
  "status": 200
}
```

Logs should be centrally searchable.

46. Reliability

Initial availability target:

99.5%

Growth target:

99.9%

Later:

99.95%+

Requirements:

- Health checks
- Graceful shutdown
- Automatic restart
- Timeouts
- Retry policies
- Circuit breakers
- Idempotency
- Dead-letter queues
- Database backups

47. Performance Targets

Initial targets:

Area	Target
API p50	<150 ms
API p95	<500 ms
Feed p95	<700 ms
Search p95	<500 ms
Typical DB query	<100 ms
Cached response	<100 ms

AI and voice will have separate latency budgets because external AI processing is inherently variable.

48. Database Scaling

Initial:

Single PostgreSQL instance/cluster

Growth:

Primary
|
+-- Read Replica

Later:

Domain-specific database ownership

Only introduce database separation when required.

49. Caching Strategy

Cache:

Popular experiences
Popular destinations
Experience details
Places
Feed candidates
Recommendation results
Static configuration

Do not blindly cache:

Payment state
Booking state
Critical transactional state

Use explicit TTLs and invalidation rules.

50. Data Lifecycle

Core product data:

PostgreSQL

Cached data:

Redis

Media:

Cloudinary

Analytics/event data:

Event pipeline

AI-specific temporary context:

AI storage / Redis / appropriate persistent store

Optional document workloads:

MongoDB

only when justified.

51. Analytics Architecture

Track:

Acquisition
Activation
Engagement
Retention
Trips
Bookings
Revenue
Ads
Affiliate conversions
AI usage
Voice usage

Important product funnel:

```
Install
↓
Signup
↓
Onboarding
↓
First Feed Interaction
↓
Save
↓
Trip Creation
↓
Booking
↓
Repeat Visit
```

52. AI Personalization Data

The platform should build a user preference representation from:

```
Explicit preferences
+
Onboarding
+
Behavior
+
Search
+
Saved content
+
Trips
+
Bookings
+
AI conversations
+
Voice interactions
```

The system should distinguish between:

Explicit preference

User directly says:

I like trekking.

Inferred preference

User repeatedly saves trekking experiences.

Both can influence recommendations, but they should remain distinguishable internally.

53. Privacy & Data Governance

The platform should minimize unnecessary collection.

Sensitive data must have:

- clear purpose
- access control
- retention policy
- deletion strategy

AI systems should not receive unnecessary personal information.

Analytics identifiers should be designed to minimize exposure of raw personal data.

54. Future Microservice Extraction

Microservices are not part of the initial deployment requirement.

Potential extraction order:

First

```
AI Service
Voice Service
Notification Service
```

Second

```
Recommendation Service
Search Service
```

Third

Trip Service
Experience Service
Booking Service

Later if justified

User Service
Payment Service
Ads Service

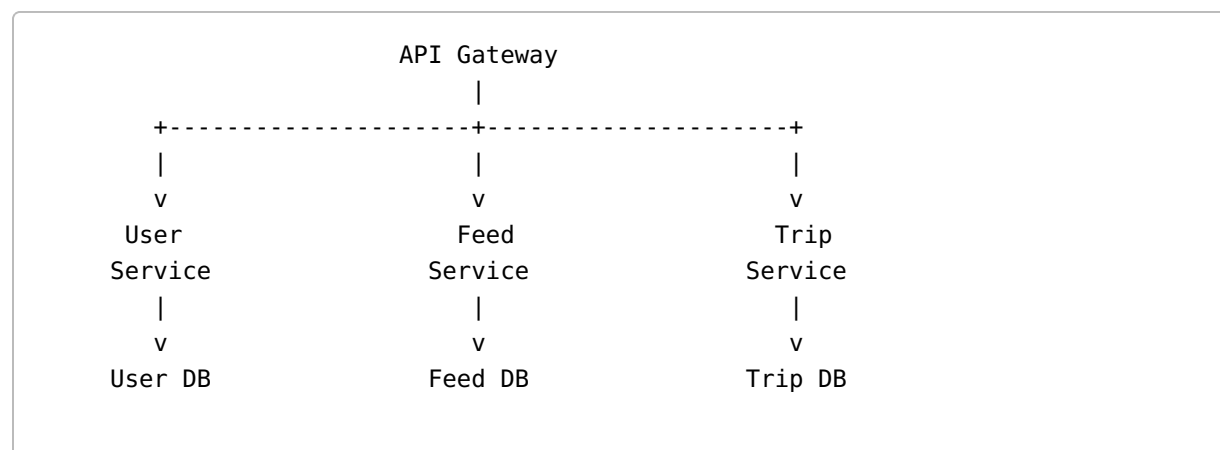
55. Microservice Extraction Criteria

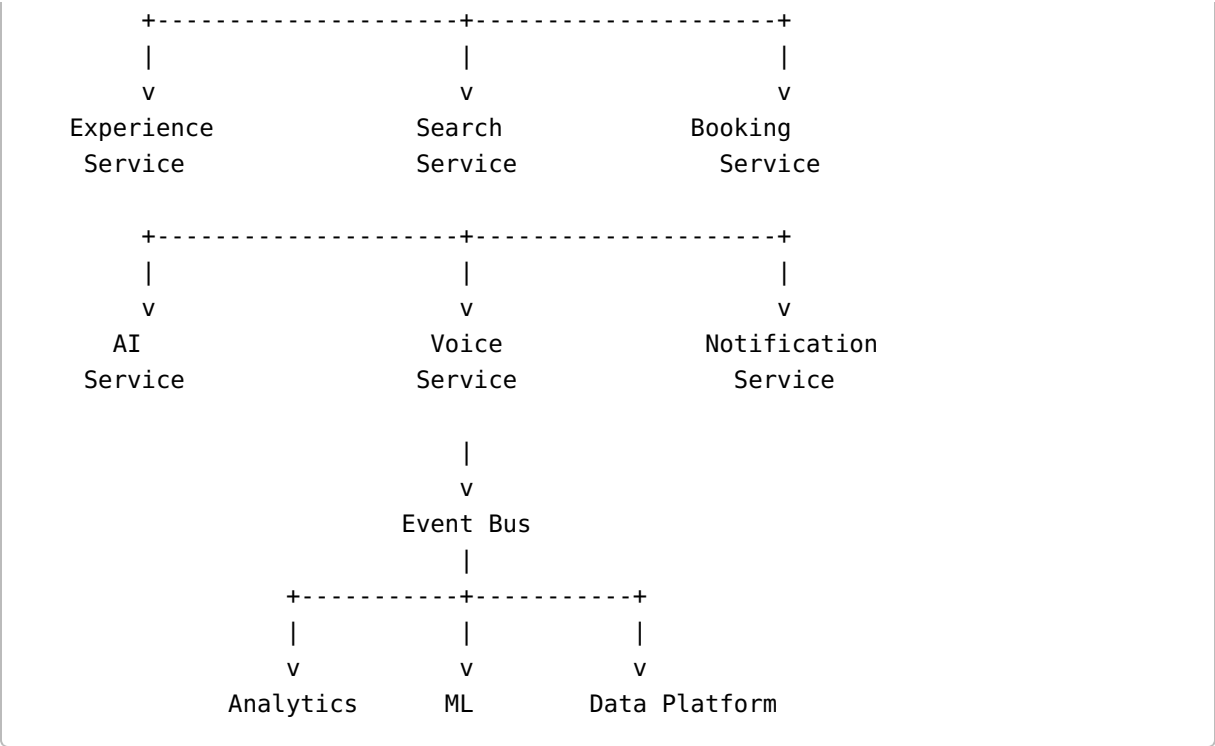
A module becomes a service only when one or more conditions are met:

1. Independent scaling requirement
2. Failure isolation requirement
3. Different technology requirement
4. Independent deployment requirement
5. Independent team ownership
6. Significant performance bottleneck
7. Data isolation requirement

Do not extract services merely because a diagram looks cleaner.

56. Target Future Architecture

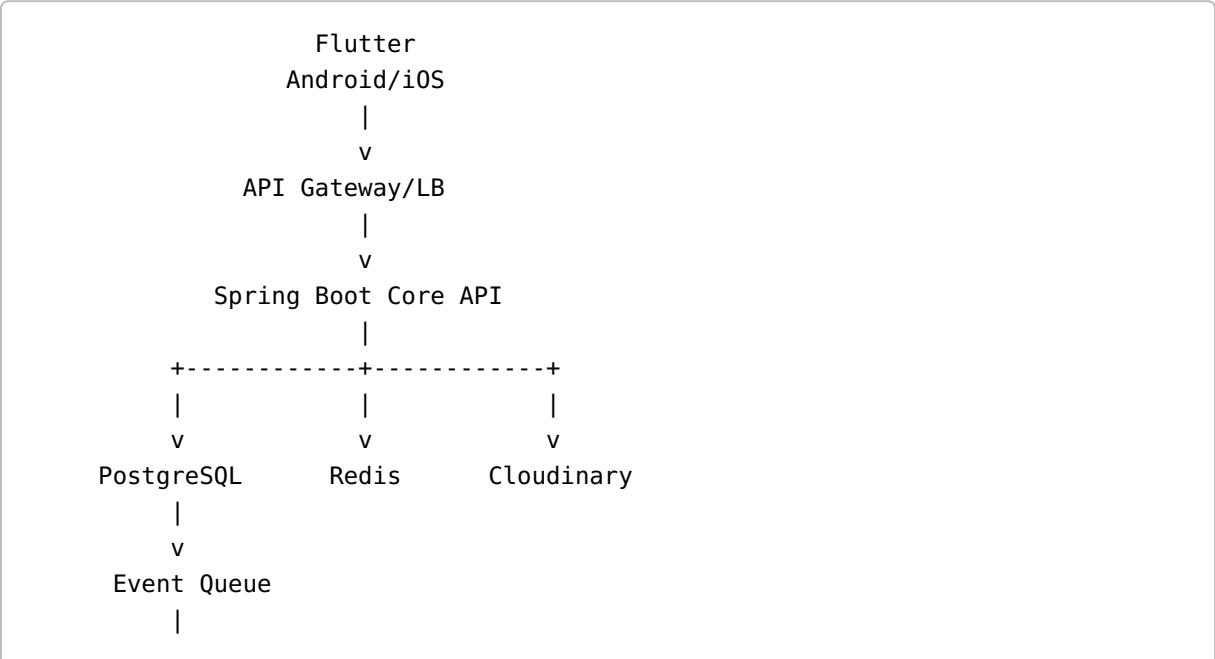


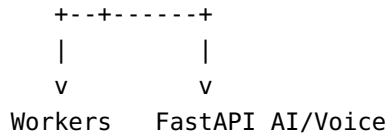


This is the **future evolution**, not the V1 requirement.

57. V1 Deployment Model

The initial production system should remain operationally simple.





There is no requirement for Kubernetes, Kafka, service mesh, or dozens of services at launch.

58. Local Development

Local environment:

Docker Compose

```

├─ PostgreSQL
├─ Redis
├─ Spring Boot
├─ FastAPI AI
├─ FastAPI Voice
└─ Workers

```

Flutter runs through the standard Flutter development environment.

59. Configuration Management

Separate:

```

development
staging
production

```

configuration.

Never hard-code:

- API keys
- database passwords
- cloud credentials
- AI provider keys
- payment secrets

Use environment variables locally and managed secrets in production.

60. API Communication

Flutter → Spring Boot:

HTTPS REST

Spring Boot → AI:

Internal HTTP API

Voice:

Streaming/WebSocket/WebRTC-style communication

depending on the selected voice provider and implementation.

Asynchronous workloads:

Queue/Event Bus

61. External Provider Abstraction

External dependencies should be hidden behind interfaces.

Examples:

```
MapsProvider  
PaymentProvider  
BookingProvider  
AIProvider  
SpeechProvider  
MediaProvider
```

This allows provider replacement without rewriting the product domain.

62. Cost Optimization

Early architecture should optimize for product development rather than maximum infrastructure complexity.

Avoid initially:

- Kubernetes
- Kafka cluster
- Service mesh
- Multiple databases
- Multiple cloud providers
- Dedicated infrastructure for every module

Use managed infrastructure wherever practical.

63. Architecture Decision Summary

Decision	V1 Choice
Mobile	Flutter
Core Backend	Java + Spring Boot
Backend Pattern	Modular Monolith
AI	Python + FastAPI
Voice	Python + FastAPI
Primary DB	PostgreSQL
Geospatial	PostGIS
Cache	Redis
Media	Cloudinary
MongoDB	Optional / only if justified
Search	PostgreSQL initially
Advanced Search	OpenSearch later
Events	Managed queue initially
Streaming Events	Kafka later

Decision	V1 Choice
Cloud	AWS
Containers	Docker
Initial Runtime	ECS
Kubernetes	Later if justified
CI/CD	GitHub Actions
Monitoring	OpenTelemetry + AWS/metrics stack
Mobile Push	FCM + APNs
Maps	Google Maps abstraction
Payments	Provider abstraction
Architecture Evolution	Modular Monolith → Selective Microservices

64. Engineering Non-Negotiables

1. Core backend remains modular.
2. Modules must have clear ownership.
3. No uncontrolled cross-module repository access.
4. PostgreSQL remains the source of truth.
5. Redis is not the primary database.
6. MongoDB is optional.
7. AI cannot bypass business rules.
8. Voice cannot block normal API infrastructure.
9. External providers must be abstracted.
10. Application servers should remain stateless.
11. Important asynchronous work should use queues.
12. Important user actions should generate events.
13. Feed should use cursor pagination.
14. Secrets must never be committed to source control.
15. Production must have monitoring and backups.
16. Microservices should only be created when justified by measurable requirements.

65. Development Phases

Phase 1 — Foundation

Build:

Flutter foundation
Spring Boot foundation
PostgreSQL
Redis
Authentication
AWS environment
Docker
CI/CD
Logging
Monitoring

Phase 2 — Core Product

Build:

Onboarding
User profile
Preferences
Home
Feed
Experiences
Places
Search
Trips
Itineraries
Reminders
Maps

Phase 3 — Intelligence

Build:

Behavioral events
Recommendation engine
Personalized feed
AI assistant
AI memory/context
Semantic search

Phase 4 — Voice

Build:

- Voice interface
- STT
- AI orchestration
- Tool calling
- TTS
- Voice trip planning
- Voice reminders

Phase 5 — Monetization

Build:

- Affiliate integrations
- Bookings
- Payments
- Contextual ads
- Sponsored experiences
- Revenue analytics

Phase 6 — Scale

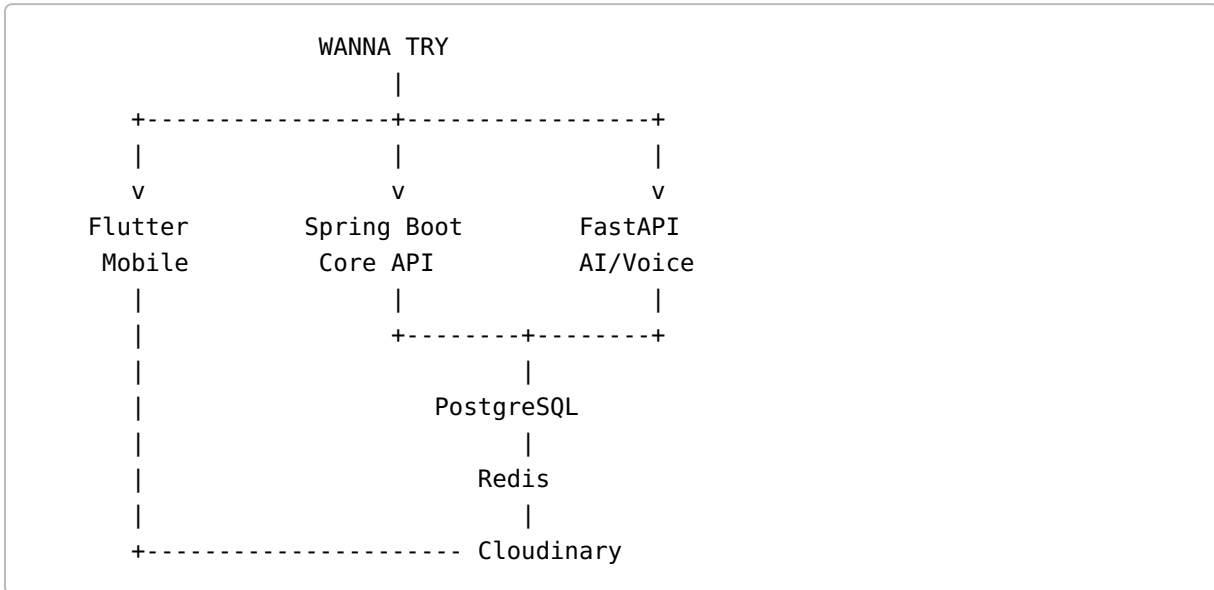
Evaluate:

- OpenSearch
- Kafka
- Dedicated recommendation service
- Notification service
- AI service scaling
- Voice service scaling
- Read replicas
- Data warehouse

66. Final Architecture Decision

Wanna Try V1 will **not** be launched as a traditional collection of independent microservices.

It will be a **modular product platform** containing:



The core backend is one **Spring Boot modular monolith**.

AI and voice are separate runtimes because they have fundamentally different workloads and technology requirements.

The entire platform is developed as one coordinated monorepo initially.

As Wanna Try grows, individual modules can be extracted into microservices based on actual scaling and operational requirements.

67. Final Principle

The architecture is designed around one rule:

Do not optimize for millions of users before having the first thousand. Build the system so that reaching millions is possible without having to rewrite the product.

This architecture gives Wanna Try:

- Fast initial development

- Strong separation of concerns
- Scalable core backend
- Dedicated AI/voice capability
- Strong personalization foundation
- Geographic capabilities
- Reliable media handling
- Clear monetization integration
- Low initial infrastructure complexity
- A controlled path to microservices
- Android + iOS support
- Production-grade observability and security

V1 target architecture:

Flutter + Spring Boot Modular Monolith + Python/FastAPI AI/Voice + PostgreSQL/PostGIS + Redis + Cloudinary + AWS

with **MongoDB, OpenSearch, Kafka, and independent microservices introduced only when there is a demonstrated need.**